EE443 Neural Networks Final Report

A Comparative Study of Convolutional and Transformer-Based Neural Networks on CIFAR-10

Emre Oral 22003050

Section 1

Group 31

1)Abstract

This project is about comparing two types of intelligence models for recognizing images. We are looking at neural networks and transformer-based architectures to see which one is better at classifying pictures in the CIFAR-10 dataset. We picked three models. ResNet-18, DenseNet-121 and DeiT-Tiny. And trained them from the beginning under the same conditions. Then we compared these image classification models in terms of how they can classify images how long it takes to train them how efficiently they use parameters and how well they perform for each class of images. What we found out is that convolutional neural networks are better than transformer-based architectures at recognizing images and they also work faster. Convolutional neural networks like ResNet-18 and DenseNet-121 are more accurate and efficient than transformer-based models, like DeiT-Tiny. ResNet-18 achieves the best overall performance, while DenseNet-121 provides a competitive alternative with fewer parameters. In contrast, DeiT-Tiny exhibits lower accuracy and significantly longer training time, highlighting the limitations of transformer-based approaches when applied to small-scale image datasets without large-scale pretraining.

2)Introduction

Image classification has long been a central problem in computer vision and has driven many of the major advances in deep learning. Early approaches relied on handcrafted features combined with classical classifiers, but the introduction of convolutional neural networks (CNNs) fundamentally changed the field by enabling end-to-end learning of hierarchical visual representations.

CNN architectures such as AlexNet, VGG, ResNet, and DenseNet have achieved remarkable success by exploiting local spatial correlations through convolutional filters. Among these, ResNet introduced residual connections that enabled the training of much deeper networks, while DenseNet further improved feature reuse by densely connecting layers.

In the few years people have started using a new kind of system for looking at pictures. This system is called Vision Transformers or ViT for short. Vision Transformers work in a way than the old systems. They do not just look at parts of the picture like the old systems did. Instead Vision Transformers look at the picture and see how all the parts are connected. This is similar to how some computers understand language.

There is another system called Data- Image Transformers or DeiT for short. Data-efficient Image Transformers are similar to Vision Transformers. They also look at the picture and see how all the parts are connected.. Data-efficient Image Transformers can learn from fewer pictures, than Vision Transformers. This is because Data-efficient Image Transformers have some ways of learning that do not need as many pictures. Vision Transformers and Data-efficient Image Transformers are both used for vision tasks.

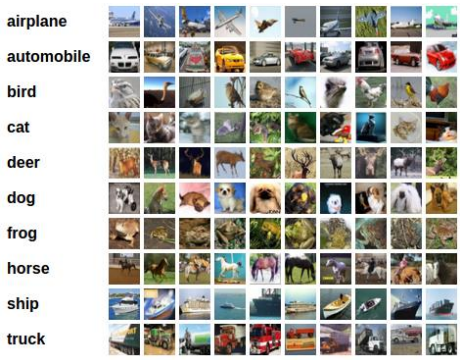
The transformer-based models have done well with big datasets, like ImageNet.. Can these models work with smaller datasets? That is still something people are trying to figure out. CIFAR-10 is an example to look at because it is small and has low-quality images.

The primary objective of this project is to systematically compare CNN-based and transformer-based models on CIFAR-10. By training all models from scratch under identical conditions, we aim to analyze their performance, computational cost, and robustness across different classes, thereby providing insights into the suitability of each architecture for small-scale image classification tasks.

3)Methods

Dataset Description

The CIFAR-10 dataset consists of 60,000 color images of size 32×32 pixels, divided into ten classes representing common objects and animals. Each class contains 6,000 images, with 50,000 images used for training and 10,000 images reserved for testing. The dataset is widely used as a benchmark due to its balanced class distribution and moderate difficulty.



To support model selection and prevent overfitting, a validation set is created by randomly splitting 10% of the training data. This validation set is used for model selection and early performance assessment during training.

Data Preprocessing and Augmentation

Getting the data ready and adding more to it is really important to make the models work better especially when we do not have a lot of data. For the models that use CNN like ResNet-18 and DenseNet-121 we keep the pictures the size they are which is 32×32. When we are training the models we do things like take a part of the picture and flip it sideways to make the data more different. We make all the pictures look the same by using the standard deviation of the CIFAR-10 pictures.

The Transformer-based models like DeiT-Tiny need pictures to work properly. This is because they look at the picture, in parts. So we make the CIFAR-10 images bigger to 224×224 before we put them into the model. Making the pictures bigger uses computer power but we have to do it so that the Transformer-based model works correctly. The Transformer-based models need this to work the way they are supposed to.

The same normalization statistics are applied across all models to ensure a fair comparison.

Model Architectures

ResNet-18

ResNet-18 is a residual convolutional neural network consisting of 18 layers. The key idea behind ResNet is the use of skip connections that allow the network to learn residual mappings instead of direct transformations. This design significantly alleviates the vanishing gradient problem and enables stable training of deeper networks.

DenseNet-121

Each layer of the densely linked CNN DenseNet-121 gets as inputs the outputs of every layer that came before it. This extensive interdependence improves gradient flow, lowers the number of parameters, and promotes feature reuse. In situations where parameter efficiency is crucial, DenseNet is especially appealing.

DeiT-Tiny

DeiT-Tiny is a lightweight vision transformer that models global relationships by using self-attention mechanisms and processing images as sequences of fixed-size patches. Transformers learn interactions across all patches rather than depending on locality priors like CNNs do. This may be less useful when training data is few, even if it can be useful on huge datasets.

All models are trained from scratch without pretrained weights to ensure that the observed performance differences are due solely to architectural characteristics.

Training Procedure

The cross-entropy loss function is used to train all models. Because it works well for training both CNNs and transformers, the AdamW optimizer is used. The learning rate is progressively lowered over 15 epochs using a cosine annealing learning rate scheduler.

When a GPU is available, training is done on it. After every epoch on the validation set, the model's performance is assessed, and the model with the highest validation accuracy is chosen for additional testing.

5)Results

Overall Performance Comparison

Table 1 summarizes the test accuracy, number of trainable parameters, and total training time for each model.

Model Name	Test Accuracy	Parameters	Training Time (minutes)
ResNet-18	**0.7568**	11.18M	**8.5**
DenseNet-121	0.7539	6.96M	10.7
DeiT-Tiny	0.7052	5.53M	59.1

Table 1: Overall Model Performance

ResNet-18 achieves the highest test accuracy while also requiring the least training time. DenseNet-121 performs comparably in terms of accuracy but is slightly slower. DeiT-Tiny shows a significant drop in accuracy and requires substantially more training time.

Class-Level Analysis

Confusion matrices for each model are presented in Figures 2–4.

```
Confusion Matrix:
 [[853  17  36  13  27   0  10   1  35   8]
 [ 24 890   4   7   2   2   9   2  30  30]
 [ 66   1 670  69  75  38  55  14   7   5]
 [ 29   8  41 640  98  79  62  23  13   7]
 [ 24   2  65  32 814  13  35   8   6   1]
 [ 18   4  39 233  80 554  34  33   3   2]
 [  8   5  31  46  40   9 850   3   7   1]
 [ 29   3  22  43 128  37   8 725   3   2]
 [ 75  16  11   8   6   2   3   1 868  10]
 [ 76 127   8  16   3   3  14  13  36 704]]
```

Figure 2: Confusion matrix for ResNet-18 on the CIFAR-10 test set.

```
Confusion Matrix:
 [[824  11  46  17   5   4   4   5  67  17]
 [ 27 839   7  10   0   6   7   2  37  65]
 [ 64   0 701  75  46  44  30  23  12   5]
 [ 21   7  71 641  32 110  42  36  23  17]
 [ 30   3  87  56 666  43  44  42  26   3]
 [ 22   3  55 200  28 629  13  36  10   4]
 [ 18   1  56  65  23  27 794   3  10   3]
 [ 32   1  34  51  44  54   3 763   6  12]
 [ 59  14   8   8   1   2   3   1 892  12]
 [ 59  73   8  11   1   5   4   7  42 790]]
```

Figure 3: Confusion matrix for DenseNet-121 on the CIFAR-10 test set.

```
Confusion Matrix:
 [[726  44  37  10  11   2  12  15  97  46]
 [ 29 775   1   6   1   2   8  14  12 152]
 [ 78   8 559  72 110  21  80  43  18  11]
 [ 29  14  35 611  65  82  71  50  19  24]
 [ 15   3  68  41 724  14  52  71   9   3]
 [ 13   9  45 280  64 439  31  99  13   7]
 [ 15   7  33  46  55   8 816   8   8   4]
 [ 23   2  35  39  84  32   6 759   3  17]
 [ 67  35  12  11   9   1   4   4 816  41]
 [ 33  71   1  12   4   2   7  15  28 827]]
```

Figure 4: Confusion matrix for DeiT-Tiny on the CIFAR-10 test set.

Visually comparable classes, such cat and dog or deer and horse, show greater confusion rates across all models. Stronger diagonal dominance is a sign of improved class discriminating in CNN-based models. DeiT-Tiny shows a higher incidence of misclassification across animal classes, indicating that it has trouble collecting fine-grained local traits.

Precision, Recall, and F1-Score Evaluation

Detailed precision, recall, and F1-score metrics for each class are reported in Table 2.

	precision	recall	f1-score	support
0	0.7097	0.8530	0.7748	1000
1	0.8295	0.8900	0.8587	1000
2	0.7228	0.6700	0.6954	1000
3	0.5781	0.6400	0.6075	1000
4	0.6394	0.8140	0.7162	1000
5	0.7517	0.5540	0.6379	1000
6	0.7870	0.8500	0.8173	1000
7	0.8809	0.7250	0.7954	1000
8	0.8611	0.8680	0.8645	1000
9	0.9143	0.7040	0.7955	1000
accuracy			0.7568	10000
macro avg	0.7674	0.7568	0.7563	10000
weighted avg	0.7674	0.7568	0.7563	10000

	precision	recall	f1-score	support
0	0.7128	0.8240	0.7644	1000
1	0.8813	0.8390	0.8596	1000
2	0.6533	0.7010	0.6763	1000
3	0.5653	0.6410	0.6007	1000
4	0.7872	0.6660	0.7216	1000
5	0.6807	0.6290	0.6538	1000
6	0.8411	0.7940	0.8169	1000
7	0.8312	0.7630	0.7956	1000
8	0.7929	0.8920	0.8395	1000
9	0.8513	0.7900	0.8195	1000
accuracy			0.7539	10000
macro avg	0.7597	0.7539	0.7548	10000
weighted avg	0.7597	0.7539	0.7548	10000

	precision	recall	f1-score	support
0	0.7062	0.7260	0.7160	1000
1	0.8006	0.7750	0.7876	1000
2	0.6768	0.5590	0.6123	1000
3	0.5417	0.6110	0.5742	1000
4	0.6424	0.7240	0.6808	1000
5	0.7280	0.4390	0.5477	1000
6	0.7507	0.8160	0.7820	1000
7	0.7041	0.7590	0.7305	1000
8	0.7977	0.8160	0.8067	1000
9	0.7306	0.8270	0.7758	1000
accuracy			0.7052	10000
macro avg	0.7079	0.7052	0.7014	10000
weighted avg	0.7079	0.7052	0.7014	10000

Table 2: Per-class precision, recall, and F1-score for all evaluated models(ResNet,DenseNet,Deit accordingly.).

Across the majority of classes, ResNet-18 and DenseNet-121 consistently demonstrate higher recall, especially for item categories with strong local textures. DeiT-Tiny's poorer overall accuracy is partly due to its decreased recall for a number of classes.

Computational Efficiency

DeiT-Tiny has the fewest parameters, but because of its greater input resolution and self-attention calculations, it takes a lot longer to train. This outcome demonstrates that the number of parameters is not a reliable measure of computing efficiency.

For CIFAR-10-sized datasets, CNN-based models provide a more advantageous trade-off between accuracy and computational cost.

6)Discussion

CNN-based architectures continue to be quite successful for small-scale image classification applications, according to the experimental data. ResNet-18 is especially well-suited for real-world applications with constrained resources because it provides the optimum balance between accuracy and processing economy.

DenseNet-121 illustrates the advantages of dense connection and feature reuse by achieving comparable accuracy with fewer parameters. Nevertheless, some of these benefits are countered by the higher computing expense.

Conversely, despite having fewer parameters, DeiT-Tiny performs worse than both CNN models. When transformer-based models are trained from scratch on tiny datasets, this outcome reveals a fundamental weakness. Large data sets and significant inductive biases brought forth by pretraining and data augmentation are crucial components of transformers.

These findings suggest that while transformers are powerful tools for large-scale vision tasks, CNNs remain the preferred choice for datasets like CIFAR-10. Future work could explore hybrid architectures, stronger augmentation strategies, or the use of pretrained transformer models to mitigate these limitations.

7)References

[1] A. Krizhevsky, "Learning multiple layers of features from tiny images," *Master's thesis*, University of Toronto, Toronto, ON, Canada, 2009.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[3] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 4700–4708.

[4] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021, pp. 10347–10357.

8)Appendix(Terminal Output)

```
Device: cuda

========================================
Model: resnet18
========================================
100%|[redacted]| 170M/170M [00:03<00:00, 45.1MB/s]
Trainable parameters: 11181642
Epoch 01 | Train Acc: 0.4010 | Val Acc: 0.4848
Epoch 02 | Train Acc: 0.5265 | Val Acc: 0.5580
Epoch 03 | Train Acc: 0.5851 | Val Acc: 0.5994
Epoch 04 | Train Acc: 0.6288 | Val Acc: 0.6178
Epoch 05 | Train Acc: 0.6556 | Val Acc: 0.6544
Epoch 06 | Train Acc: 0.6796 | Val Acc: 0.6602
Epoch 07 | Train Acc: 0.6953 | Val Acc: 0.6894
Epoch 08 | Train Acc: 0.7124 | Val Acc: 0.6974
Epoch 09 | Train Acc: 0.7226 | Val Acc: 0.7028
Epoch 10 | Train Acc: 0.7372 | Val Acc: 0.7164
Epoch 11 | Train Acc: 0.7499 | Val Acc: 0.7306
Epoch 12 | Train Acc: 0.7572 | Val Acc: 0.7210
Epoch 13 | Train Acc: 0.7684 | Val Acc: 0.7212
Epoch 14 | Train Acc: 0.7765 | Val Acc: 0.7380
Epoch 15 | Train Acc: 0.7815 | Val Acc: 0.7420

Test Accuracy: 0.7568
Training Time (min): 8.481731883684795
Confusion Matrix:
 [[853  17  36  13  27   0  10   1  35   8]
 [ 24 890   4   7   2   2   9   2  30  30]
 [ 66   1 670  69  75  38  55  14   7   5]
 [ 28   3  41 640  98  79  62  23  13   7]
 [ 24   2  65  32 814  13  35   8   6   1]
 [ 18   4  39 233  80 554  34  33   3   2]
 [  8   5  31  46  40   9 850   3   7   1]
 [ 29   3  22  43 128  37   8 725   3   2]
 [ 75  16  11   8   6   2   3   1 868  10]
 [ 76 127   8  16   3   3  14  13  36 704]]
              precision    recall  f1-score   support

           0       0.7097    0.8530    0.7748      1000
           1       0.8295    0.8900    0.8587      1000
           2       0.7228    0.6700    0.6954      1000
           3       0.5781    0.6400    0.6075      1000
           4       0.6394    0.8140    0.7162      1000
           5       0.7517    0.5540    0.6379      1000
           6       0.7870    0.8500    0.8173      1000
           7       0.8809    0.7250    0.7954      1000
           8       0.8611    0.8680    0.8645      1000
           9       0.9143    0.7040    0.7955      1000

    accuracy                           0.7568     10000
   macro avg       0.7674    0.7568    0.7563     10000
weighted avg       0.7674    0.7568    0.7563     10000


========================================
Model: densenet121
========================================
Trainable parameters: 6964106
Epoch 01 | Train Acc: 0.3893 | Val Acc: 0.4630
Epoch 02 | Train Acc: 0.5073 | Val Acc: 0.5362
Epoch 03 | Train Acc: 0.5618 | Val Acc: 0.5754
Epoch 04 | Train Acc: 0.6008 | Val Acc: 0.6016
Epoch 05 | Train Acc: 0.6326 | Val Acc: 0.6226
Epoch 06 | Train Acc: 0.6590 | Val Acc: 0.6444
Epoch 07 | Train Acc: 0.6790 | Val Acc: 0.6668
Epoch 08 | Train Acc: 0.6995 | Val Acc: 0.6848
Epoch 09 | Train Acc: 0.7162 | Val Acc: 0.7024
Epoch 10 | Train Acc: 0.7271 | Val Acc: 0.7032
Epoch 11 | Train Acc: 0.7420 | Val Acc: 0.7112
Epoch 12 | Train Acc: 0.7516 | Val Acc: 0.7176
Epoch 13 | Train Acc: 0.7656 | Val Acc: 0.7202
Epoch 14 | Train Acc: 0.7728 | Val Acc: 0.7312
Epoch 15 | Train Acc: 0.7837 | Val Acc: 0.7432

Test Accuracy: 0.7539
Training Time (min): 10.660454702377319
Confusion Matrix:
```

```
[[824  11  46  17   5   4   4   5  67  17]
 [ 27 839   7  10   0   6   7   2  37  65]
 [ 64   0 701  75  46  44  30  23  12   5]
 [ 21   7  71 641  32 110  42  36  23  17]
 [ 30   3  87  56 666  43  44  42  26   3]
 [ 22   3  55 200  28 629  13  36  10   4]
 [ 18   1  56  65  23  27 794   3  10   3]
 [ 32   1  34  51  44  54   3 763   6  12]
 [ 58  14   8   8   1   2   3   1 892  12]
 [ 59  73   8  11   1   5   4   7  42 790]]
              precision    recall  f1-score   support

           0       0.7128    0.8240    0.7644      1000
           1       0.8813    0.8390    0.8596      1000
           2       0.6533    0.7010    0.6763      1000
           3       0.5653    0.6410    0.6007      1000
           4       0.7872    0.6660    0.7216      1000
           5       0.6807    0.6290    0.6538      1000
           6       0.8411    0.7940    0.8169      1000
           7       0.8312    0.7630    0.7956      1000
           8       0.7929    0.8920    0.8395      1000
           9       0.8513    0.7900    0.8195      1000

    accuracy                           0.7539     10000
   macro avg       0.7597    0.7539    0.7548     10000
weighted avg       0.7597    0.7539    0.7548     10000

========================================
Model: deit_tiny
========================================
Trainable parameters: 5526346
Epoch 01 | Train Acc: 0.3365 | Val Acc: 0.4220
Epoch 02 | Train Acc: 0.4522 | Val Acc: 0.4882
Epoch 03 | Train Acc: 0.5044 | Val Acc: 0.5338
Epoch 04 | Train Acc: 0.5394 | Val Acc: 0.5288
Epoch 05 | Train Acc: 0.5708 | Val Acc: 0.5810
Epoch 06 | Train Acc: 0.5947 | Val Acc: 0.5832
Epoch 07 | Train Acc: 0.6247 | Val Acc: 0.6314
Epoch 08 | Train Acc: 0.6436 | Val Acc: 0.6284
Epoch 09 | Train Acc: 0.6631 | Val Acc: 0.6604
Epoch 10 | Train Acc: 0.6771 | Val Acc: 0.6510
Epoch 11 | Train Acc: 0.6920 | Val Acc: 0.6794
Epoch 12 | Train Acc: 0.7087 | Val Acc: 0.6768
Epoch 13 | Train Acc: 0.7207 | Val Acc: 0.6908
Epoch 14 | Train Acc: 0.7316 | Val Acc: 0.6816
Epoch 15 | Train Acc: 0.7416 | Val Acc: 0.7124
Test Accuracy: 0.7052
Training Time (min): 59.13555173873901
Confusion Matrix:
[[726  44  37  10  11   2  12  15  97  46]
 [ 29 775   1   6   1   2   8  14  12 152]
 [ 78   8 559  72 110  21  80  43  18  11]
 [ 29  14  35 611  65  82  71  50  19  24]
 [ 15   3  68  41 724  14  52  71   9   3]
 [ 13   9  45 280  64 439  31  99  13   7]
 [ 15   7  33  46  55   8 816   8   8   4]
 [ 23   2  35  39  84  32   6 759   3  17]
 [ 67  35  12  11   9   1   4   4 816  41]
 [ 32  71   1  12   4   2   7  15  28 827]]
              precision    recall  f1-score   support

           0       0.7062    0.7260    0.7160      1000
           1       0.8006    0.7750    0.7876      1000
           2       0.6768    0.5590    0.6123      1000
           3       0.5417    0.6110    0.5742      1000
           4       0.6424    0.7240    0.6808      1000
           5       0.7280    0.4390    0.5477      1000
           6       0.7507    0.8160    0.7820      1000
           7       0.7041    0.7590    0.7305      1000
           8       0.7977    0.8160    0.8067      1000
           9       0.7306    0.8270    0.7758      1000
```